



Software
Performance
Engineering

LECTURE 4

Assembly Language & Computer Architecture

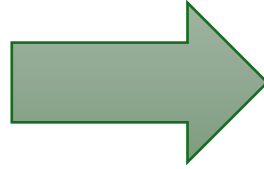
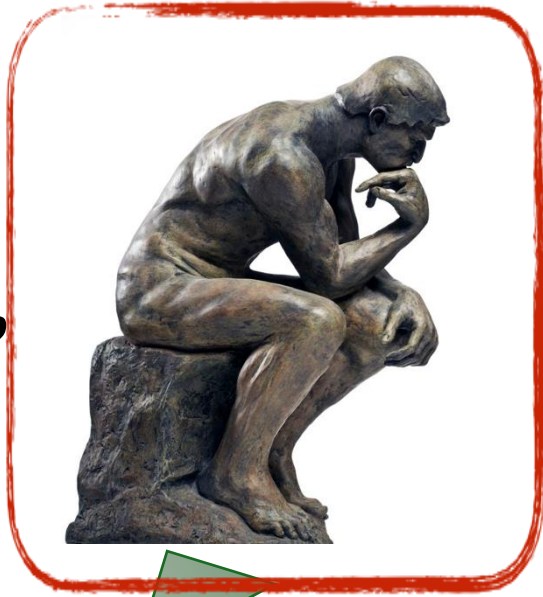
Xuhao Chen

September 22, 2026



Performance Engineering Loop

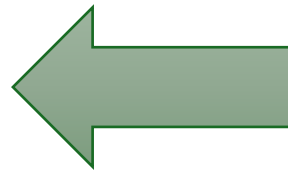
think,



code,



run,



measure,



Approaches to making code faster

- **Algorithm:** e.g., $n^2 \rightarrow n \log n$
- **Work:** Any redundant work in my code? **Bentley's Rules**
- **Locality:** Actually memory instead of compute bound? **Caches**
- **Parallelization:** can we split work into independent tasks? **Cilk**
- **SIMD:** can we vectorize expensive operations? e.g., **AVX2**
 - A certain type of parallelization

Overview

- Lecture 1: Performance is the currency of computing
- Lecture 2: Bit hacks → word **parallelism**
- Lecture 3: Bentley's rules → reducing **work** (beyond algorithm)
- Lecture 4: this lecture → **understanding** your hardware
- Lecture 5: Vectorization → data **parallelism**
- Lecture 6 (& 8, 9, 10, 13): Multicore → task **parallelism**
- Lecture 7: C to Assembly → **understanding** your compiler
- Lecture 11, 12: cache → **locality**

Source Code to Execution

Source code fib.c

```
int64_t fib(int64_t n) {  
    if (n < 2) return n;  
    return (fib(n-1) + fib(n-2));  
}
```

```
$ clang fib.c -o fib
```

Compilation

Four stages {
Preprocessing
Compiling
Assembling
Linking

Machine code fib

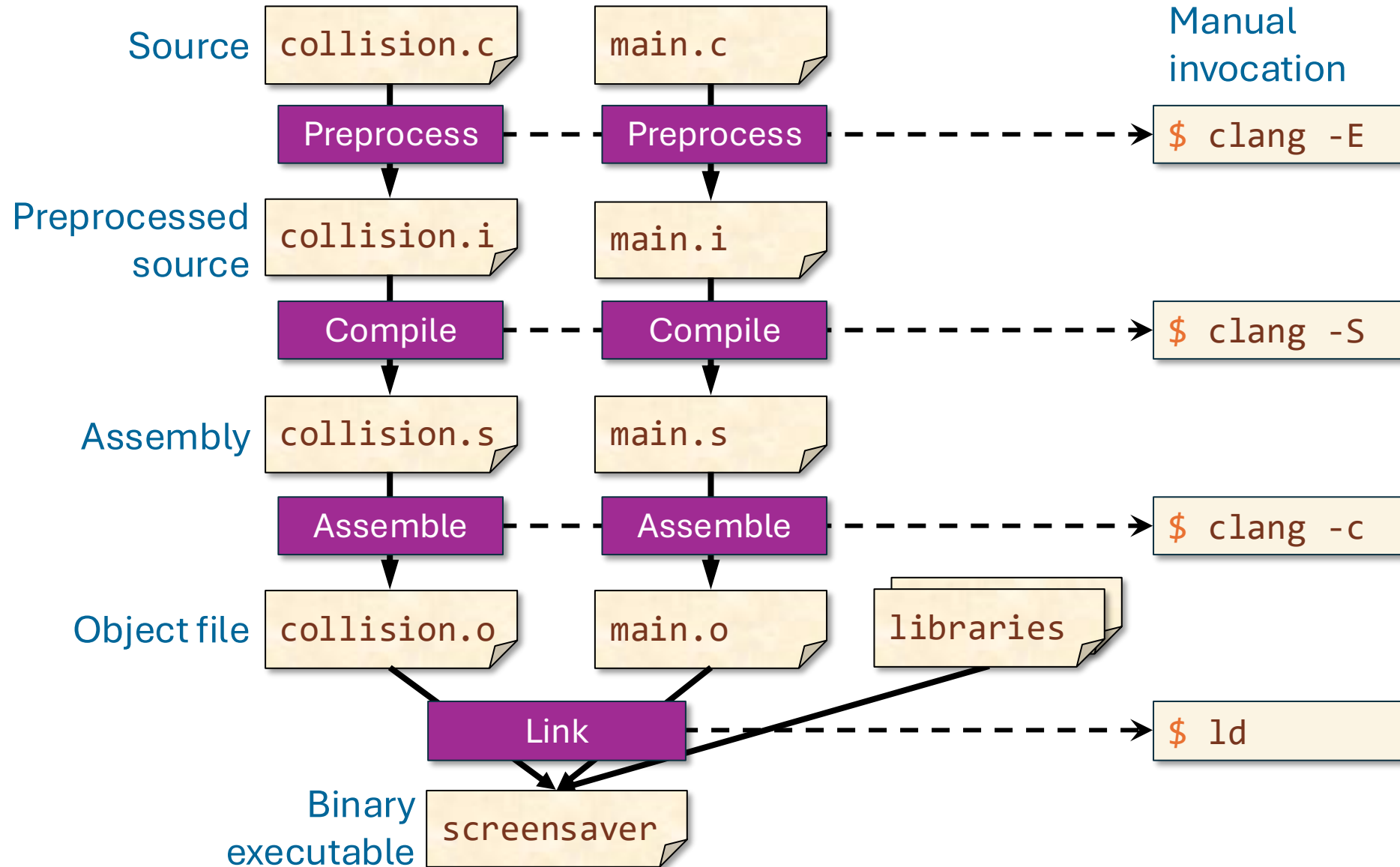
```
01010101 01001000 10001001  
11100101 01010011 01001000  
10000011 11101100 00001000  
10001001 01111101 11110100  
10000011 01111101 11110100  
00000001 01111111 00001000  
10001011 01000101 11110100  
10001001 01000101 11110000  
11101011 00011101 10001011  
01000101 11110100 10001101  
01111000 11111111 11101000  
11011011 11111111 11111111  
11111111 10001001 11000011  
10001011 01000101 11110100  
10001101 01111000 11111110  
11101000 11001110 11111111  
11111111 11111111 00000001  
11000011 10001001 01011101  
11110000 10001011 01000101  
11110000 01001000 10000011  
11000100 00001000 01011011  
11001001 11000011
```

Hardware
interpretation

```
$ ./fib
```

Execution

The Four Stages of Compilation



Source Code to Assembly Code

Source code fib.c

```
int64_t fib(int64_t n) {  
    if (n < 2) return n;  
    return (fib(n-1) + fib(n-2));  
}
```

```
$ clang -O3 fib.c -S
```

Assembly code fib.s

```
        .globl    _fib  
        .p2align  4, 0x90  
_fib:   ## @fib  
        pushq    %rbp  
        movq     %rsp, %rbp  
        pushq    %r14  
        pushq    %rbx  
        movq     %rdi, %rbx  
        cmpq     $2, %rbx  
        jge      LBB0_1  
        movq     %rbx, %rax  
        jmp      LBB0_3  
  
LBB0_1:  leaq     -1(%rbx), %rdi  
        callq    _fib  
        movq     %rax, %r14  
        addq     $-2, %rbx  
        movq     %rbx, %rdi  
        callq    _fib  
        addq     %r14, %rax  
  
LBB0_3:  popq     %rbx  
        popq     %r14  
        popq     %rbp  
        retq
```

Assembly language provides a convenient symbolic representation of machine code

See <http://sourceware.org/binutils/docs/as/index.html>.

Assembly Code to Executable

Assembly code `fib.s`

```
.globl    _fib
.p2align  4, 0x90
_fib:
## @fib
pushq    %rbp
movq     %rsp, %rbp
pushq    %r14
pushq    %rbx
movq     %rdi, %rbx
cmpq     $2, %rbx
jge      LBB0_1
movq     %rbx, %rax
jmp      LBB0_3

LBB0_1:
leaq     -1(%rbx), %rdi
callq    _fib
movq     %rax, %r14
addq     $-2, %rbx
movq     %rbx, %rdi
callq    _fib
addq     %r14, %rax

LBB0_3:
popq     %rbx
popq     %r14
popq     %rbp
retq
```

Assembling

```
$ clang fib.s -o fib.o
```

Machine code

```
01010101 01001000
10001001 11100101
01010011 01001000
10000011 11101100
00001000 10001001
01111101 11110100
10000011 01111101
11110100 00000001
01111111 00001000
10001011 01000101
11110100 10001001
01000101 11110000
11101011 00011101
10001011 01000101
11110100 10001101
01111000 11111111
11101000 11011011
11111111 11111111
11111111 10001001
11000011 10001011
01000101 11110100
...
```

You can edit `fib.s` and assemble with `clang`.

Disassembling

- Binary executable **fib** with debug symbols (i.e., compiled with **-g**):

\$ objdump -S fib

Source, machine, & assembly

Disassembly of section `__TEXT,__text`:

```
_fib:
; int64_t fib(int64_t n) {
    0: 55                pushq %rbp
    1: 48 89 e5          movq %rsp, %rbp
    4: 41 56             pushq %r14
    6: 53               pushq %rbx
    7: 48 89 fb          movq %rdi, %rbx
; if (n < 2) return n;
    a: 48 83 fb 02       cmpq $2, %rbx
    e: 7d 05            jge 5 <_fib+0x15>
; }
    10: 48 89 d8          movq %rbx, %rax
    13: eb 1b            jmp 27 <_fib+0x30>
; return (fib(n-1) + fib(n-2));
    15: 48 8d 7b ff       leaq -1(%rbx), %rdi
    19: e8 e2 ff ff ff   callq -30 <_fib>
    1e: 49 89 c6          movq %rax, %r14
    21: 48 83 c3 fe       addq $-2, %rbx
    25: 48 89 df          movq %rbx, %rdi
    28: e8 d3 ff ff ff   callq -45 <_fib>
    2d: 4c 01 f0         addq %r14, %rax
; }
    30: 5b               popq %rbx
    31: 41 5e            popq %r14
    33: 5d               popq %rbp
    34: c3              retq
```

Why Assembly?

- Why bother looking at the **assembly** of your program?
 - It reveals what the compiler did and did not do
 - Bugs can arise at a low level
 - e.g., a bug in code might only have an effect when compiling at **-O3**
 - Sometimes the compiler is the source of the bug!
 - You can write or modify the assembly by hand, when all else fails
- **Reverse engineering**: You can decipher what a program does when you only have access to its binary

Expectations of Students

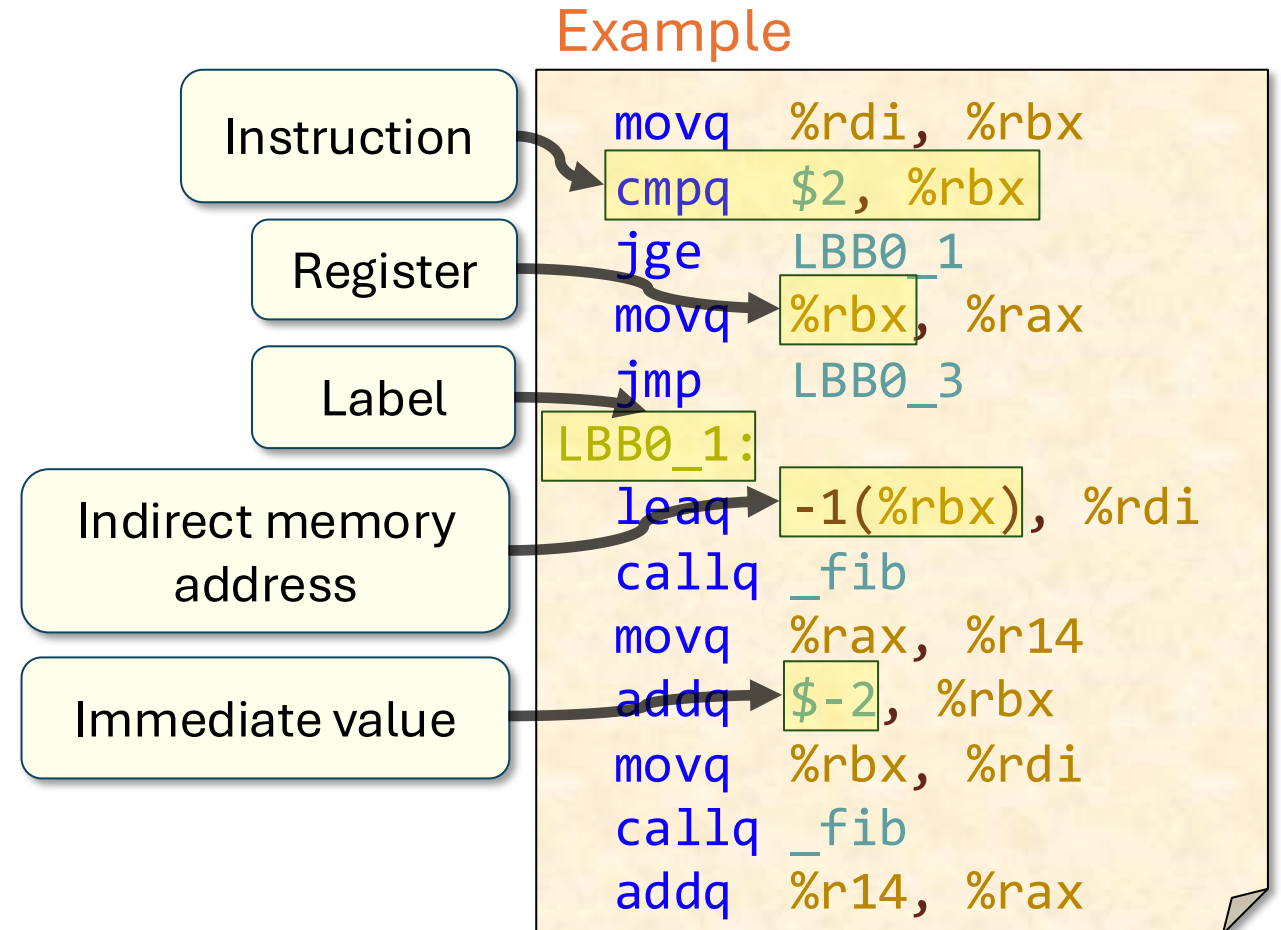
- Assembly is complicated, and you needn't memorize the manual. Here's what we expect of you:
 - Understand how a compiler **implements** C language constructs using x86 instructions
 - Demonstrate a proficiency in **reading** x86 assembly language (with the aid of an architecture manual)
 - Understand the **performance implications** of common assembly patterns
 - Make simple **modifications** to x86 assembly code generated by a compiler
 - Use compiler **intrinsic functions** to use assembly instructions not directly available in C
 - Know how to go about writing your own assembly code from scratch if the situation demands it

Outline

- **x86-64 ISA Primer**
 - Opcodes
 - Operands
- **Overview of Modern Computer Architecture**

Instruction Set Architecture (ISA)

- The **ISA** specifies **syntax** and **semantics** of assembly
- Registers
- Instructions
- Data types
- Memory addressing modes



x86-64 Registers

Number	Width (bits)	Name(s)	Purpose
16	64	(many)	General-purpose registers (GPRs)
6	16	%ss, %[c-g]s	Segment registers
1	64	RFLAGS	Flags register
1	64	%rip	Instruction pointer register
7	64	%cr[0-4,8], %xcr0	Control registers
8	64	%mm[0-7]	MMX registers
1	32	mxcsr	SSE2 control register
16	128	%xmm[0-15]	XMM registers (for SSE)
	256	%ymm[0-15]	YMM registers (for AVX)
8	80	%st([0-7])	x87 FPU data registers
1	16	x87 CW	x87 FPU control register
1	16	x87 SW	x87 FPU status register
1	48		x87 FPU instruction pointer register
1	48		x87 FPU data operand pointer register
1	16		x87 FPU tag register
1	11		x87 FPU opcode register

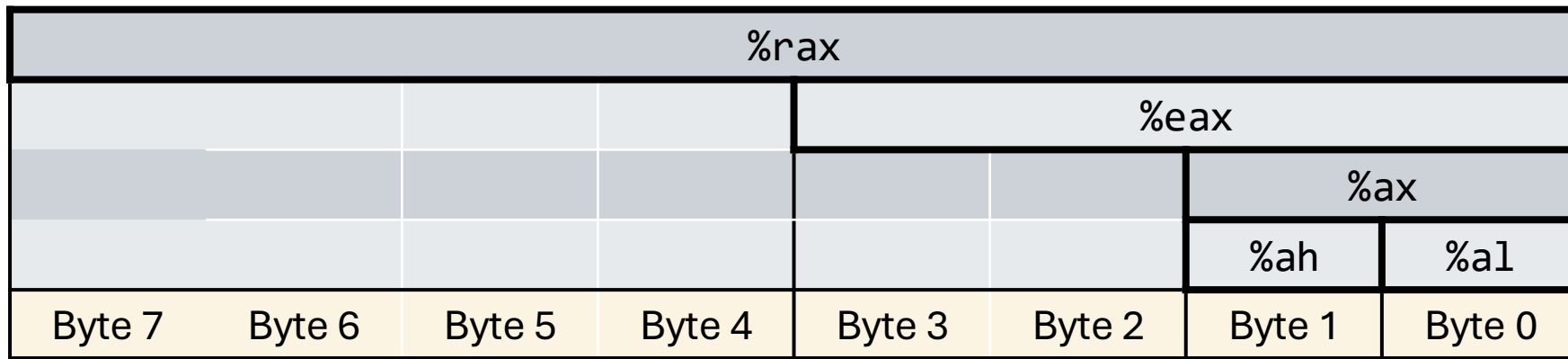
Common x86-64 Registers

Number	Width (bits)	Name(s)	Purpose
16	64	(many)	General-purpose registers (GPRs)
6	16	%ss, %[c-g]s	Segment registers
1	64	RFLAGS	Flags register
1	64	%rip	Instruction pointer register
7	64	%cr[0-4,8], %xcr0	Control registers
8	64	%mm[0-7]	MMX registers
1	32	mxcsr	SSE2 control register
16	128	%xmm[0-15]	XMM registers (for SSE)
	256	%ymm[0-15]	YMM registers (for AVX)
8	80	%st([0-7])	x87 FPU data registers
1	16	x87 CW	x87 FPU control register
1	16	x87 SW	x87 FPU status register
1	48		x87 FPU instruction pointer register
1	48		x87 FPU data operand pointer register
1	16		x87 FPU tag register
1	11		x87 FPU opcode register

x86-64 Register Aliasing

- The x86-64 general-purpose registers (GPRs) are **aliased**: each has multiple names, which refer to overlapping bytes in the register

General-purpose register layout



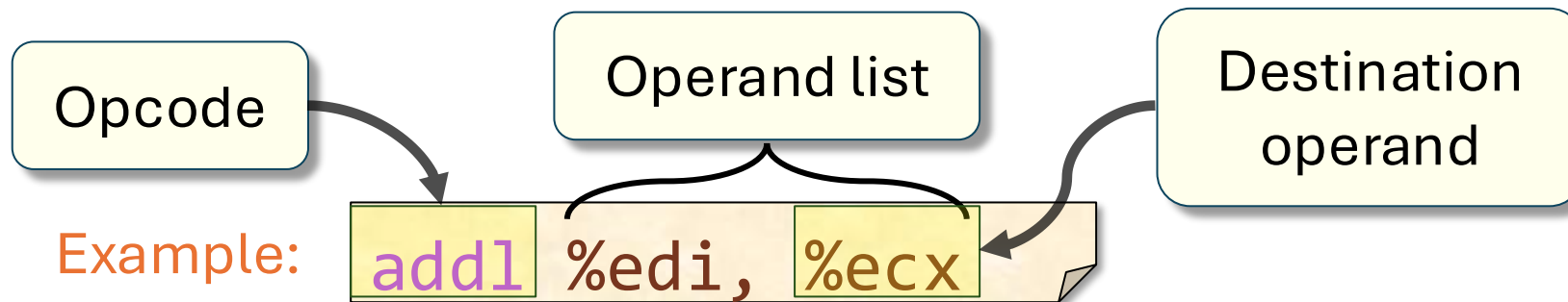
Only **%rax**, **%rbx**, **%rcx**, and **%rdx** have a separate register name for this byte

x86-64 General-Purpose Registers

64-bit name	32-bit name	16-bit name	8-bit name(s)
%rax	%eax	%ax	%ah, %al
%rbx	%ebx	%bx	%bh, %bl
%rcx	%ecx	%cx	%ch, %cl
%rdx	%edx	%dx	%dh, %dl
%rsi	%esi	%si	%sil
%rdi	%edi	%di	%dil
%rbp	%ebp	%bp	%bpl
%rsp	%esp	%sp	%spl
%r8	%r8d	%r8w	%r8b
%r9	%r9d	%r9w	%r9b
%r10	%r10d	%r10w	%r10b
%r11	%r11d	%r11w	%r11b
%r12	%r12d	%r12w	%r12b
%r13	%r13d	%r13w	%r13b
%r14	%r14d	%r14w	%r14b
%r15	%r15d	%r15w	%r15b

x86-64 Instruction Format

- Format: `<opcode> <operand_list>`
 - `<opcode>` is a short mnemonic identifying the type of instruction
 - `<operand_list>` is 0, 1, 2, or (rarely) 3 operands, separated by commas
 - For most operations, all operands are sources, and one operand might also be the destination



AT&T vs. Intel Syntax

- What does “<op> A, B” mean?

AT&T Syntax: $B \text{ <op> } A \rightarrow B$

```
movl $1, %eax
```

```
addl (%ebx,%ecx,0x2), %eax
```

```
subq 0x20(%rbx), %rax
```

Generated or used by
clang, objdump, perf,
and our lectures.

Intel Syntax: $A \leftarrow A \text{ <op> } B$

```
mov eax, 1
```

```
add eax, [ebx+ecx*2h]
```

```
sub rax, [rbx+20h]
```

Used by Intel
documentation.

Outline

- x86-64 ISA Primer
 - **Opcodes**
 - Operands
- Overview of Modern Computer Architecture

Common x86-64 Opcodes

Type of operation		Examples
Data movement	Move	mov
	Conditional move	cmov
	Sign or zero extension	movs, movz
	Stack	push, pop
Arithmetic and logic	Integer arithmetic	add, sub, mul, imul, div, idiv, lea, sal, sar, shl, shr, rol, ror, inc, dec, neg
	Binary logic	and, or, xor, not
	Boolean logic	test, cmp
Control transfer	Unconditional jump	jmp
	Conditional jumps	j<condition>
	Subroutines	call, ret

Note: The subtraction operation

“**subq** %rax, %rbx” computes $\%rbx = \%rbx - \%rax$.

Opcode Suffixes

- Opcodes might be augmented with a **suffix** that describes the **data type** of the operation or a **condition code**
- An opcode for data movement, arithmetic, or logic uses a single-character suffix to indicate the **data type**

Example

`movq -16(%rbp), %rax`

Moving a 64-bit integer

- If suffix missing, it can be inferred from the **size** of operand register

x86-64 Data Types

C declaration	C constant	x86-64 size (bytes)	Assembly suffix	x86-64 data type
char	'c'	1	b	Byte
short	106	2	w	Word
int	106	4	l or d	Double word
unsigned int	106U	4	l or d	Double word
long	106L	8	q	Quad word
unsigned long	106UL	8	q	Quad word
char *	"6.106"	8	q	Quad word
float	6.106F	4	s	Single precision
double	6.106	8	d	Double precision
long double	6.106L	16(10)	t	Extended precision

Conditional Operations

- Conditional jumps and conditional moves use a one- or two-character suffix to indicate the **condition code**

Example

```
cmpq $4096, %r14  
jne .LBB1_1
```

The jump should only be taken if the arguments of the previous comparison are **not equal**.

RFLAGS Register

Bit(s)	Abbreviation	Description
0	CF	Carry
1		Reserved
2	PF	Parity
3		Reserved
4	AF	Adjust
5		Reserved
6	ZF	Zero
7	SF	Sign
8	TF	Trap
9	IF	Interrupt enable
10	DF	Direction
11	OF	Overflow
12-63		System flags or reserved

The last ALU operation generated a carry or borrow out of the most-significant bit.

The result of the last ALU operation was 0.

The last ALU operation produced a value whose sign bit was set.

The last ALU operation resulted in arithmetic overflow.

status
register

RFLAGS Register

Bit(s)	Abbreviation	Description
0	CF	Carry
1		Reserved
2	PF	Parity
3		Reserved
4	AF	Adjust
5		Reserved
6	ZF	Zero
7	SF	Sign
8	TF	Trap
9	IF	Interrupt enable
10	DF	Direction
11	OF	Overflow
12-63		System flags or reserved

ALU operations update **status flags** in the **RFLAGS** register

Decrement **%rbx**, and set **ZF** if the result is 0.

Example:

```
decq %rbx  
jne .LBB7_1
```

Jump to label **.LBB7_1** if **ZF** is not set.

Outline

- x86-64 ISA Primer
 - Opcodes
 - **Operands**
- **Overview of Modern Computer Architecture**

x86-64 Direct Addressing Modes

- **Three** direct addressing modes

- **Immediate:** Use the specified value
- **Register:** Use the value in the specified register
- **Direct memory:** Use the value at the specified memory address

Examples

```
movq $106, %rdi
```

```
movq %rcx, %rdi
```

```
movq 0x106, %rdi
```

x86-64 Indirect Addressing Modes

- **Indirect addressing:** specifying a memory address as the result of some simple computation
 - **Register indirect:** The address is stored in the specified register
 - **Register indexed:** The address is a constant offset of the value in the specified register
 - **Instruction-pointer relative:** The address is indexed relative to `%rip`

Examples

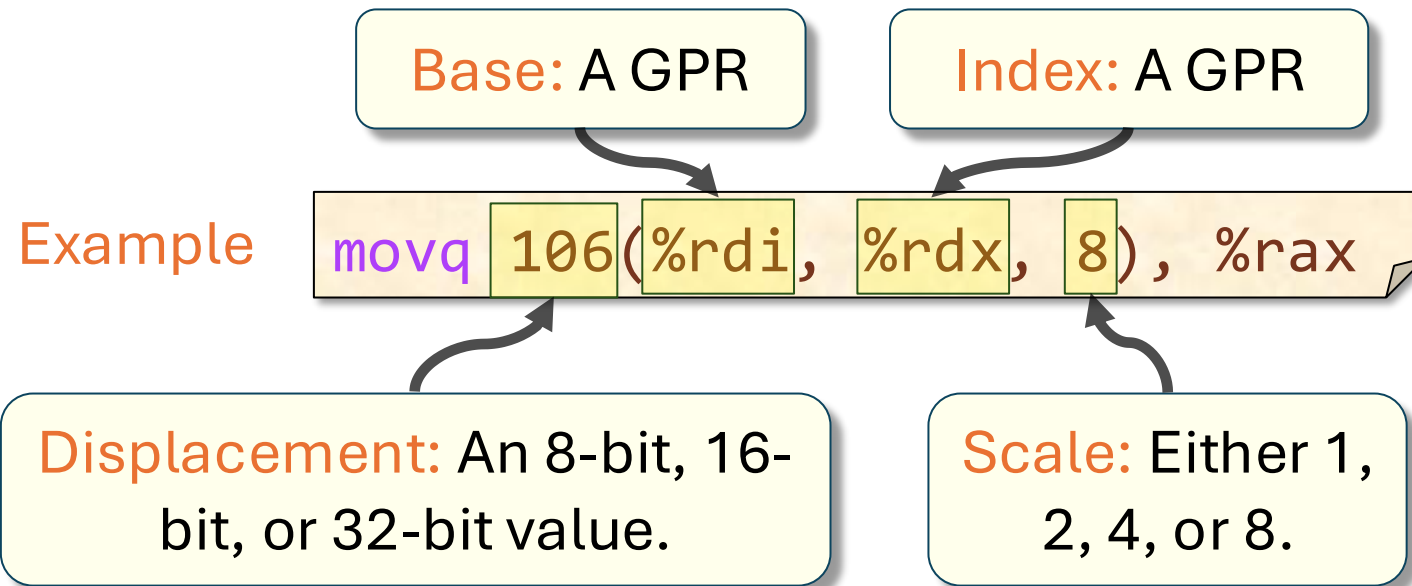
```
movq (%rax), %rdi
```

```
movq 106(%rax), %rdi
```

```
movq 106(%rip), %rdi
```

Base Indexed Scale Displacement

- Address = **Base** + **Index*****Scale** + **Displacement**



If unspecified, **Index** and **Displacement** default to **0**, and **Scale** defaults to **1**

Jump Instructions

- The jump instructions, `jmp` and `j<condition>`, take a **label** as their operand, which identifies a location in the code
 - Labels can be **symbols**, **exact addresses**, or **relative addresses**.

Example from `fib.s`

```
jge  LBB0_1
...
LBB0_1:
    leaq -1(%rbx), %rdi
```

Example from `objdump fib`

```
jge  5 <_fib+0x15>
...
15:
    leaq -1(%rbx), %rdi
```

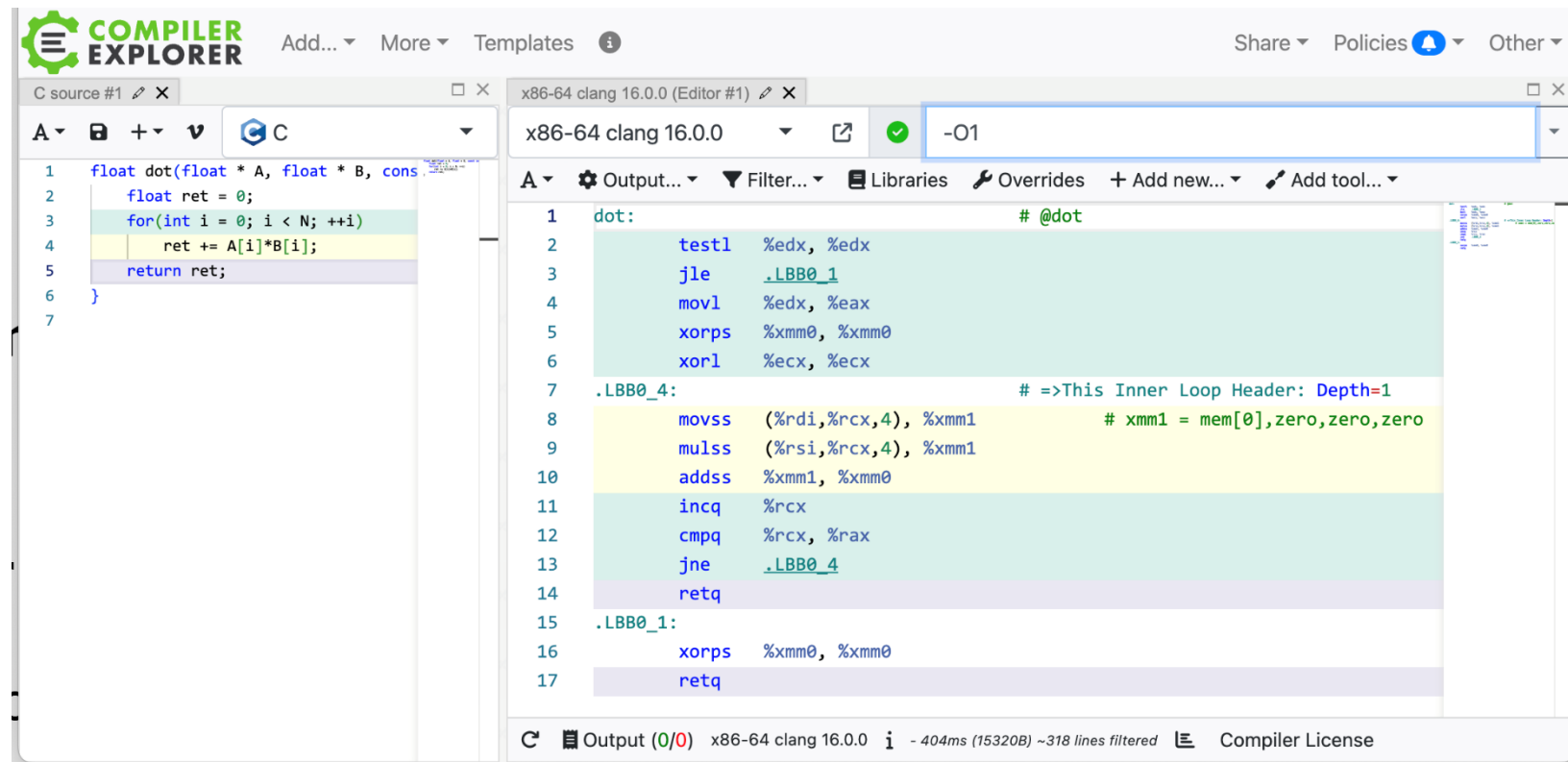
- An **indirect jump** takes as its operand an indirect address.

Example:

```
jmp  *%eax
```

Exploring Assembly Code

- Use <https://godbolt.org/>
- Select x86-64 clang 16.0.0 from pulldown
- Deselect intel asm syntax from Output



The screenshot displays the Compiler Explorer web application. On the left, the C source code for a dot product function is shown:

```
1 float dot(float * A, float * B, const int N) {
2     float ret = 0;
3     for(int i = 0; i < N; ++i)
4         ret += A[i]*B[i];
5     return ret;
6 }
7
```

On the right, the assembly output for x86-64 clang 16.0.0 is displayed. The output is filtered to show only the assembly code, and the 'Output' checkbox is unchecked. The assembly code is as follows:

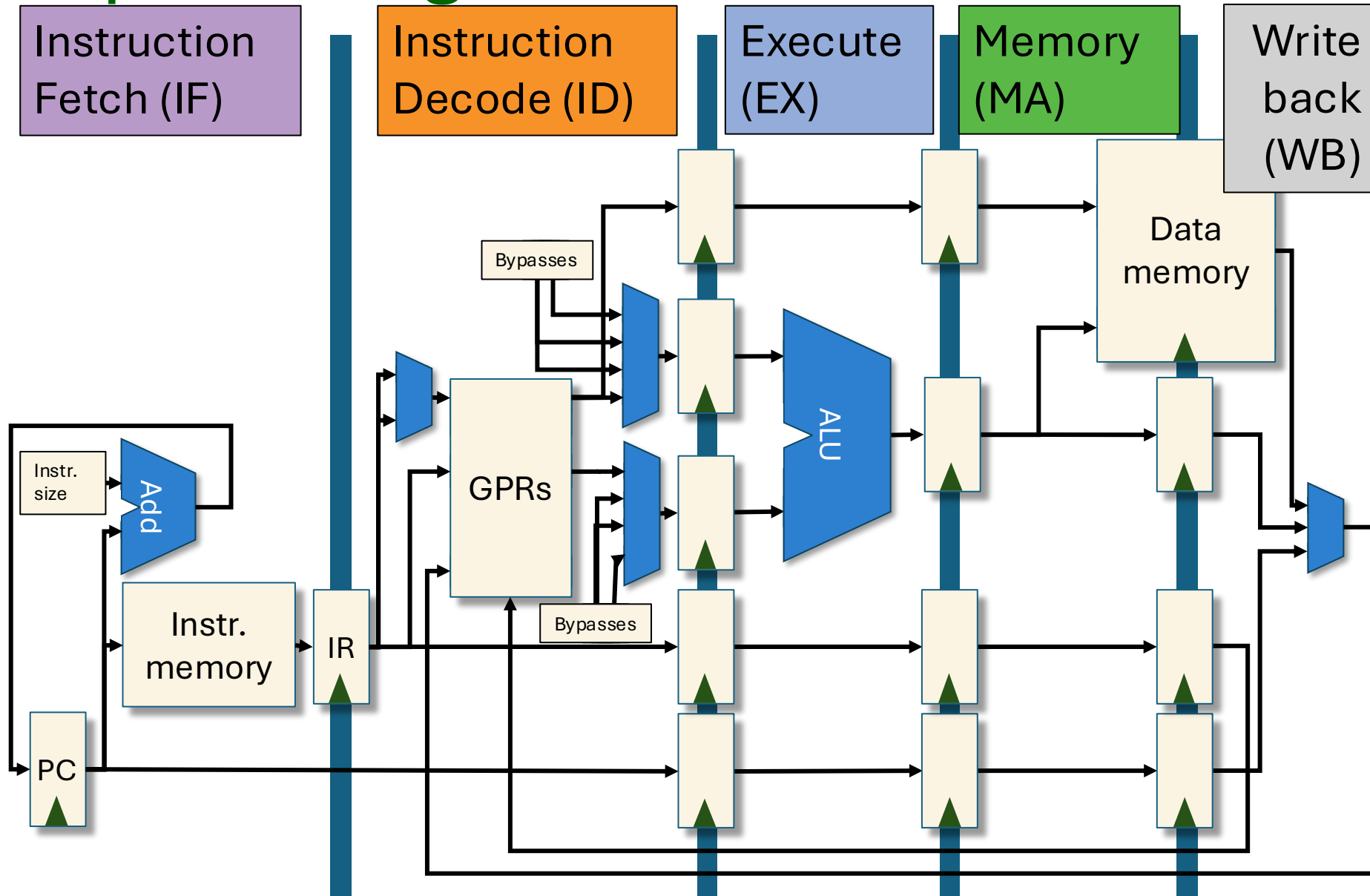
```
1 dot:                                     # @dot
2     testl   %edx, %edx
3     jle     .LBB0_1
4     movl    %edx, %eax
5     xorps   %xmm0, %xmm0
6     xorl    %ecx, %ecx
7     .LBB0_4:                             # =>This Inner Loop Header: Depth=1
8     movss   (%rdi,%rcx,4), %xmm1          # xmm1 = mem[0],zero,zero,zero
9     mulss   (%rsi,%rcx,4), %xmm1
10    addss   %xmm1, %xmm0
11    incq    %rcx
12    cmpq    %rcx, %rax
13    jne     .LBB0_4
14    retq
15 .LBB0_1:
16    xorps   %xmm0, %xmm0
17    retq
```

The status bar at the bottom indicates the output is 0/0, the compiler is x86-64 clang 16.0.0, and the execution time is -404ms (15320B) with ~318 lines filtered.

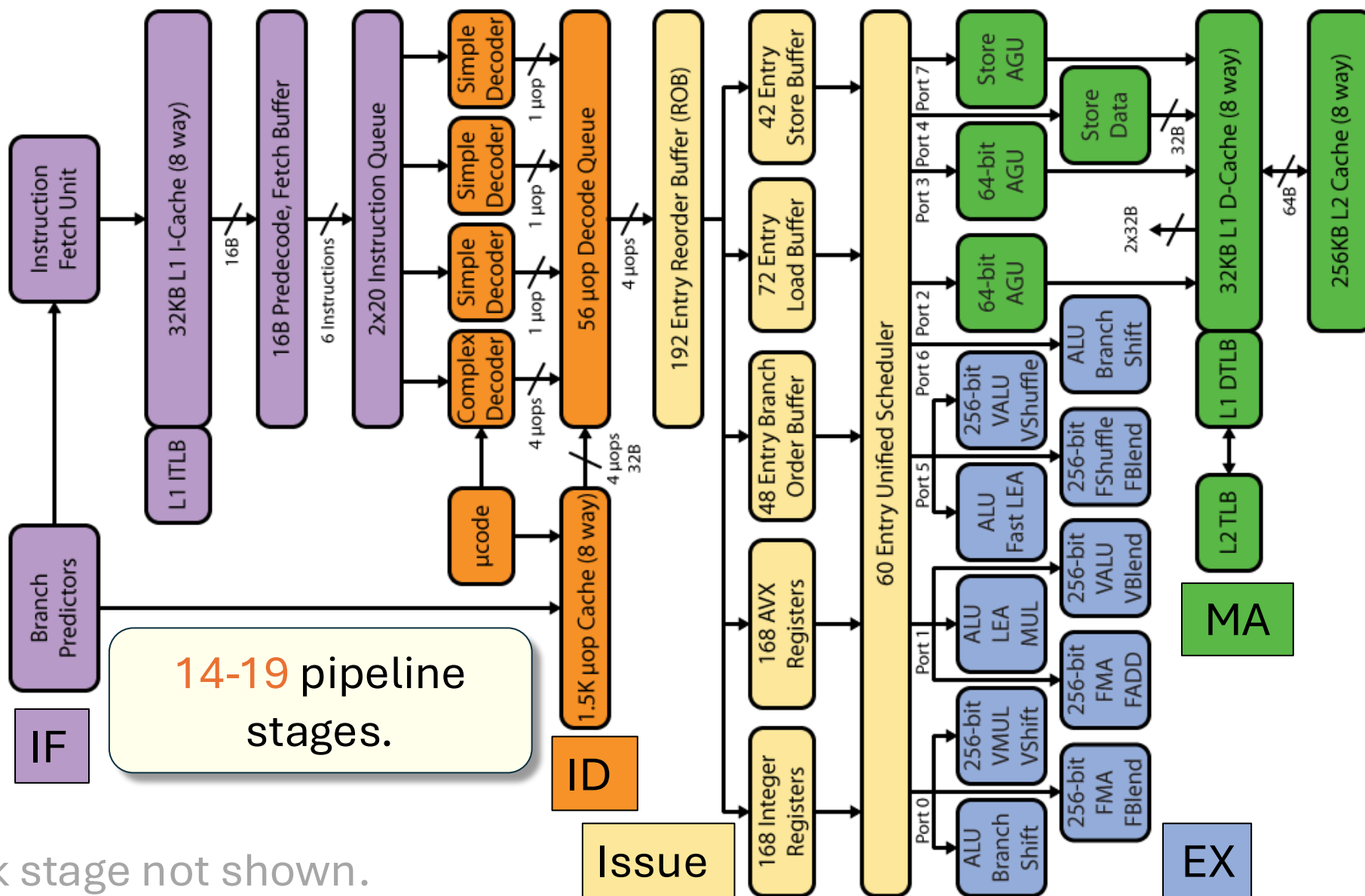
Outline

- x86-64 ISA Primer
 - Opcodes
 - Operands
- **Overview of Modern Computer Architecture**
 - Dealing with Structural Hazards
 - Dealing with Control Hazards
 - Dealing with Data Hazards

A Simple 5-Stage Processor



Intel Haswell Microarchitecture



Write-back stage not shown.

Architectural Improvements

- Two ways to improve processor performance
- Exploit **parallelism** by running multiple instructions simultaneously
 - e.g. instruction-level parallelism (ILP), vectorization, multicore
- Exploit **locality** to minimize data movement
 - e.g. caching

Pipelined Instruction Execution

- Exploit instruction-level parallelism by **executing multiple instructions simultaneously** in different pipeline stages

Ideal
^
Pipelined timing

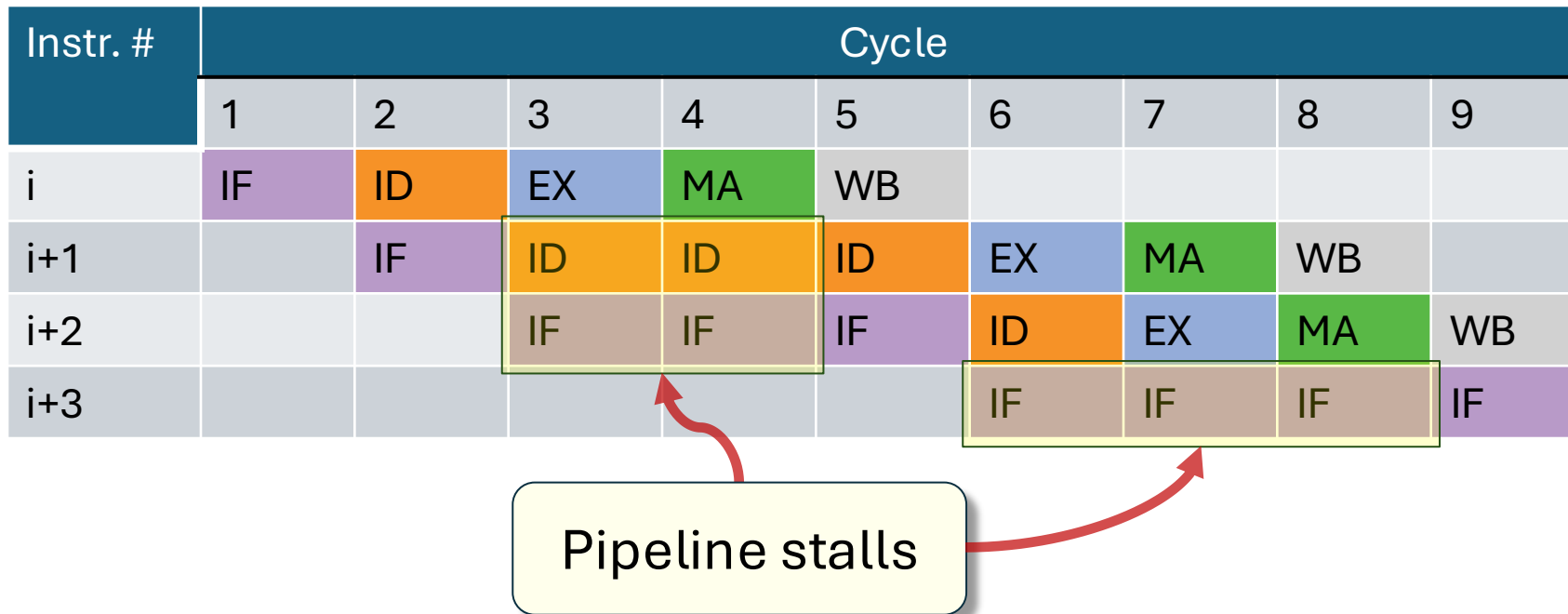
Instr. #	Cycle									
	1	2	3	4	5	6	7	8	9	10
i	IF	ID	EX	MA	WB					
i+1		IF	ID	EX	MA	WB				
i+2			IF	ID	EX	MA	WB			
i+3				IF	ID	EX	MA	WB		
i+4					IF	ID	EX	MA	WB	

Each pipeline stage is executing a different instruction.

Pipelining improves processor **throughput**.

Pipelined Execution in Practice

- In practice, various issues can prevent an instruction from executing during its designated cycle, causing the processor pipeline to **stall**



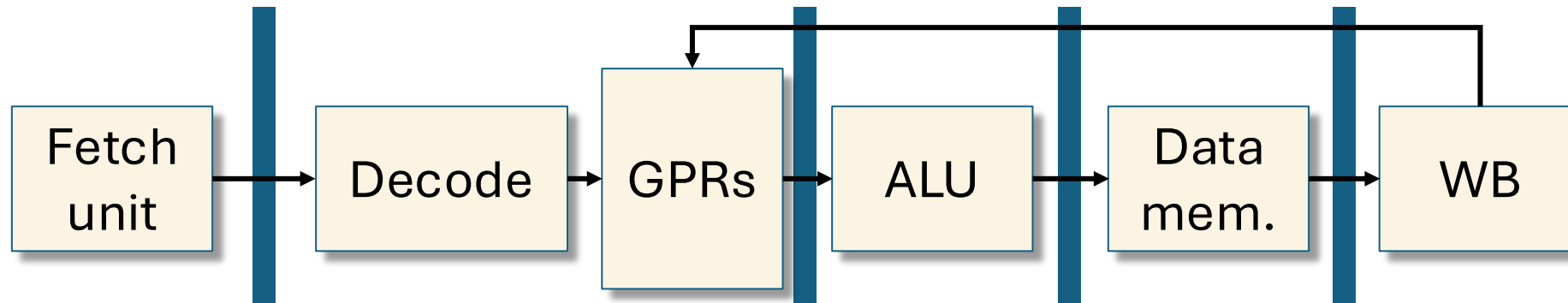
Sources of Pipeline Stalls

- Three types of **hazards** may prevent an instruction from executing during its designated clock cycle.
 - **Structural hazard**: Two instructions attempt to use the same functional unit at the same time.
 - **Control hazard**: Fetching and decoding the next instruction to execute is delayed by a decision about control flow (i.e., a conditional jump).
 - **Data hazard**: An instruction depends on the result of a prior instruction in the pipeline.

Outline

- x86-64 ISA Primer
 - Opcodes
 - Operands
- Overview of Modern Computer Architecture
 - **Dealing with Structural Hazards**
 - Dealing with Control Hazards
 - Dealing with Data Hazards

Reexamining the 5-Stage Processor



Observation: the 5-stage processor uses a **single** ALU to perform all arithmetic operations in the program.

Problem: Complex Operations

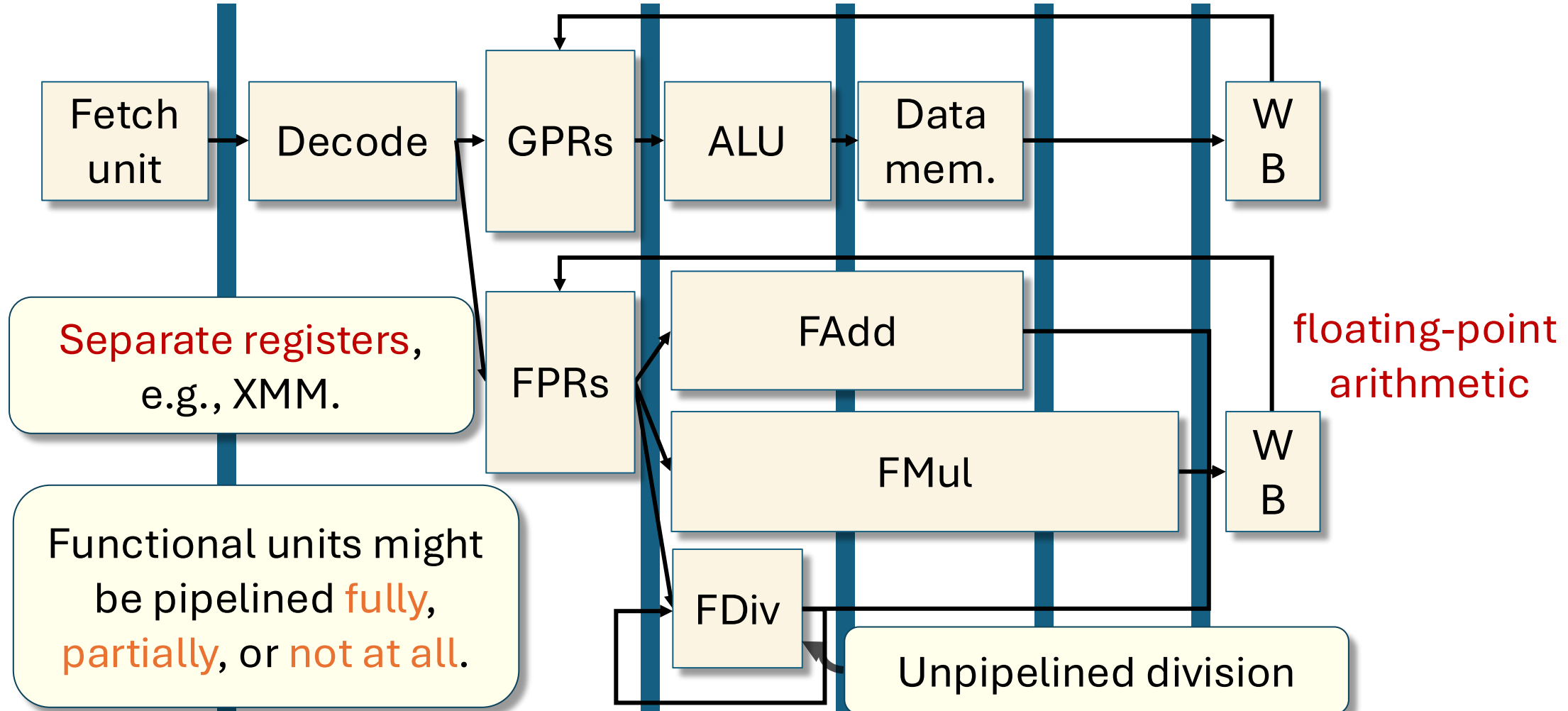
- Some arithmetic ops are **complex** to implement in HW and have **long latencies**

Operations	Example x86-64 opcodes	Latency (cycles)
Most integer arithmetic, logic, shift	add, sub, and, or, xor, sar, sal, lea...	1
Integer multiply	mul, imul	3
Integer division	div, idiv	variable
Floating-point add	addss, addsd	3
Floating-point multiply	mulss, mulsd	5
Floating-point divide	divss, divsd	variable
Floating-point fma	vfmass, vfmasd	5

How can hardware accommodate these complex operations?

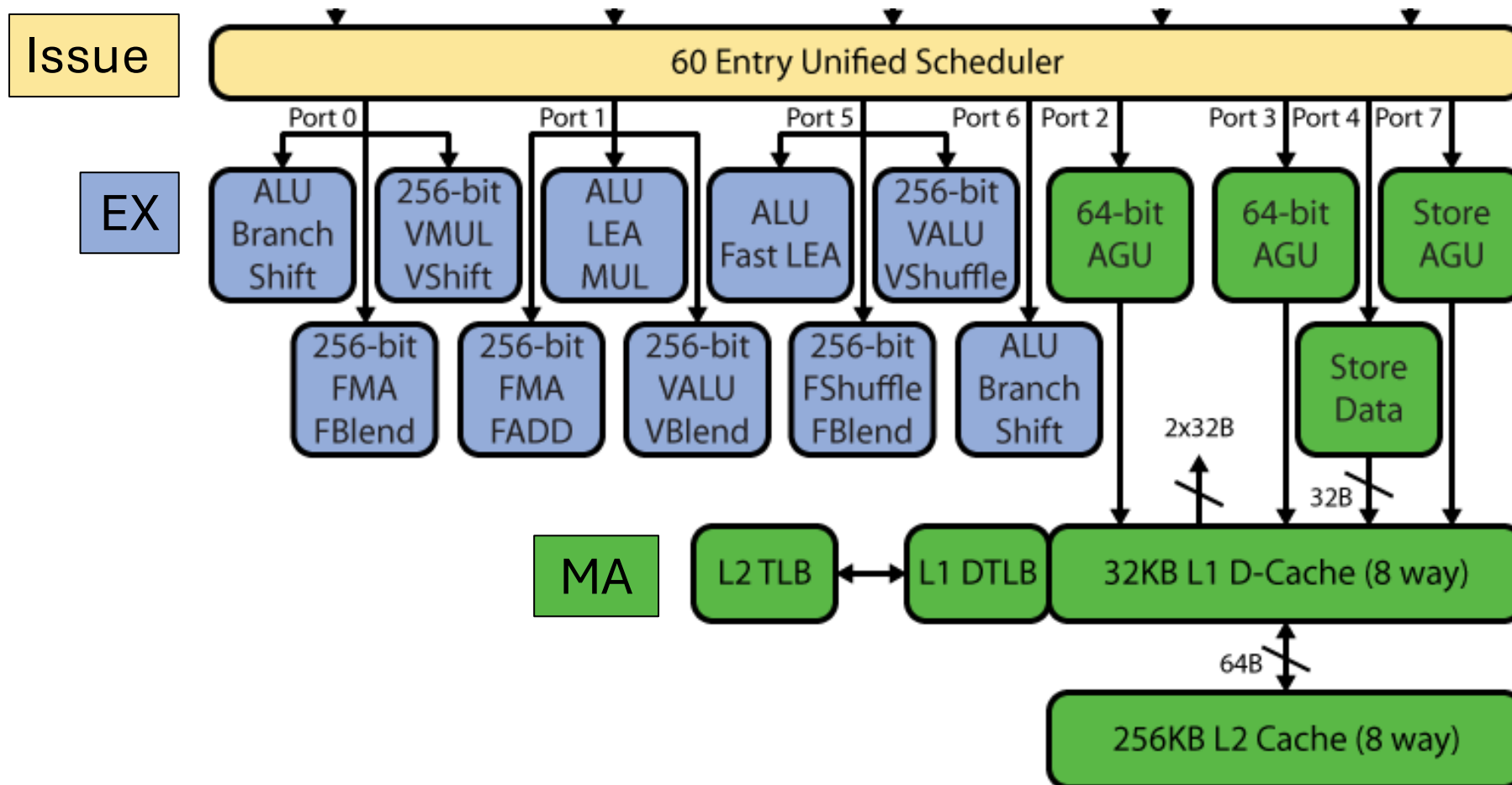
Solution: Complex Pipelining

- **Idea:** use **separate functional units** for complex operations



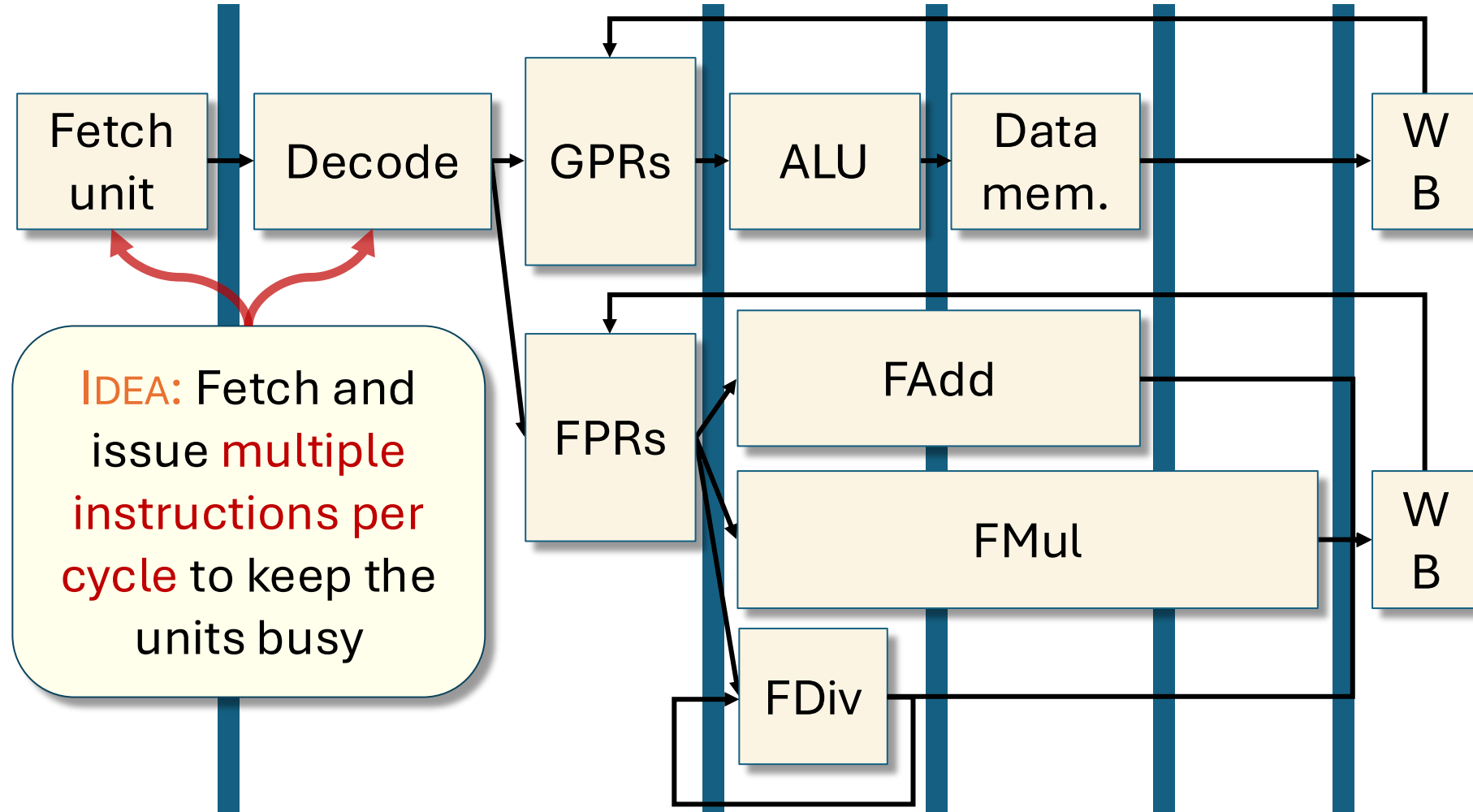
Intel Haswell Functional Units

- Haswell uses a suite of integer, vector, and floating-point functional units, distributed among **8** different **ports**



From Complex to Superscalar

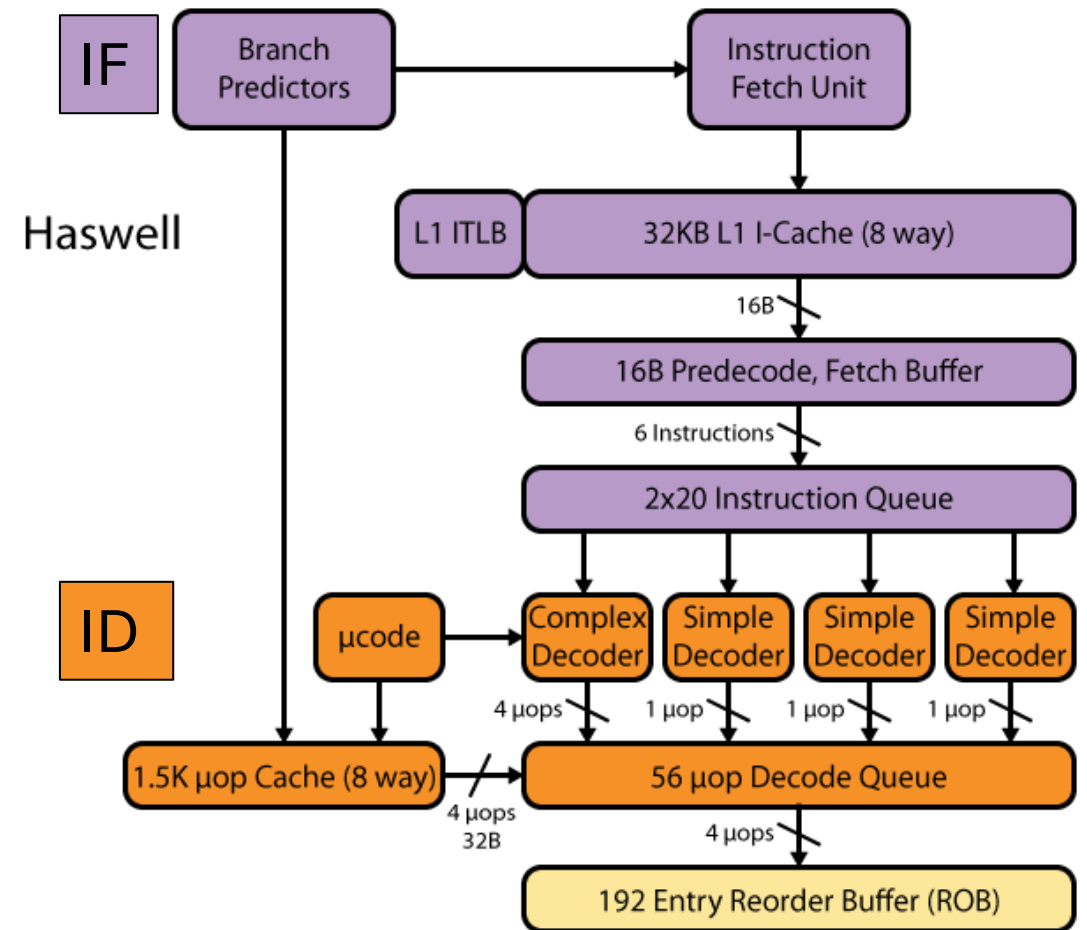
- Given additional units, how can the processor **further exploit ILP**?



Intel Haswell Fetch and Decode

- Haswell breaks up x86-64 instructions into **micro-ops**

- The fetch and decode stages can emit **4** micro-ops per cycle to the rest of the pipeline
- The fetch and decode stages implement **optimizations** on micro-op processing, including special cases for common patterns, e.g., `xor %rax, %rax`

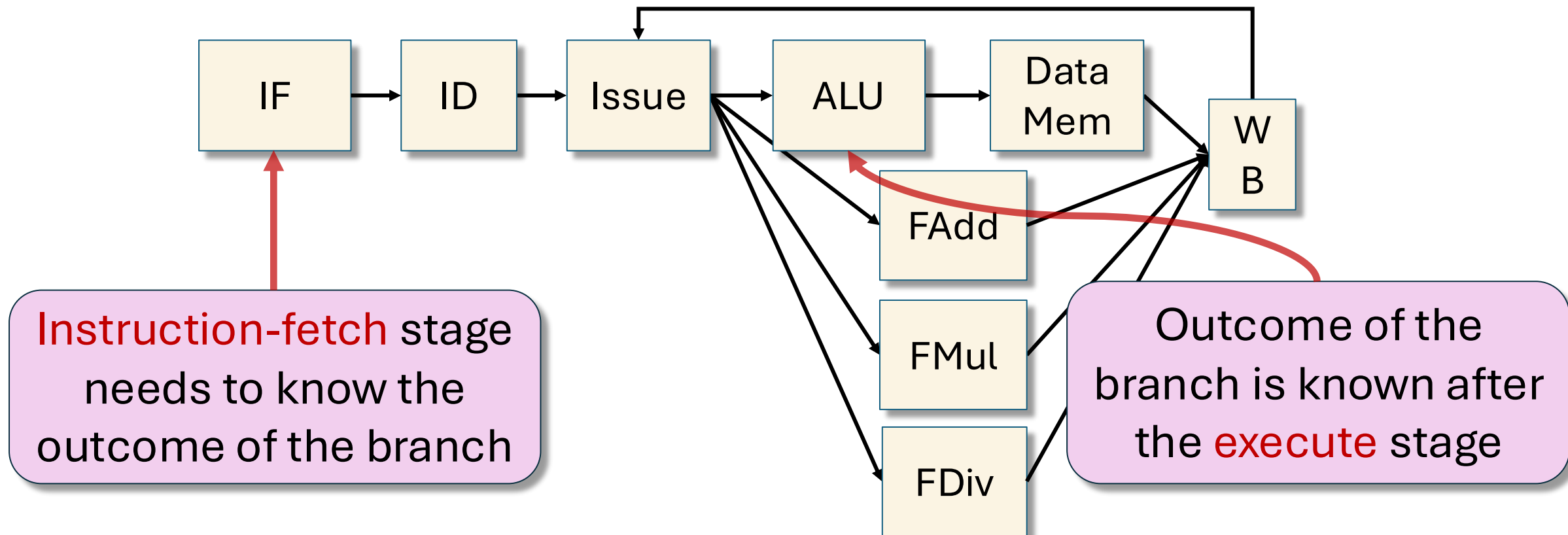


Outline

- x86-64 ISA Primer
 - Opcodes
 - Operands
- Overview of Modern Computer Architecture
 - Dealing with Structural Hazards
 - **Dealing with Control Hazards**
 - Dealing with Data Hazards

Control Hazards

- What happens if a **conditional jump**, i.e., a **branch** encountered?



Speculative Execution

- To handle a control hazard, the processor either **stalls** at the branch or **speculatively** executes past it.

Example

%rip →

```
cmpq    %r14, %rbp
jae     .LBB9_3
movq    %rbx, %rdi
movq    %rbp, %rsi
callq   bitarray_get
```

When a branch is encountered, assume it's **not taken**, and keep executing normally.

Speculative Execution

- To handle a control hazard, the processor either **stalls** at the branch or **speculatively** executes past it.

Example

`%rip` →

```
cmpq    %r14, %rbp
jae     .LBB9_3
movq    %rbx, %rdi
movq    %rbp, %rsi
callq   bitarray_get
```

When a branch is encountered, assume it's **not taken**, and keep executing normally.

If it is later discovered that the branch was **taken**, then **undo** the speculative computation.

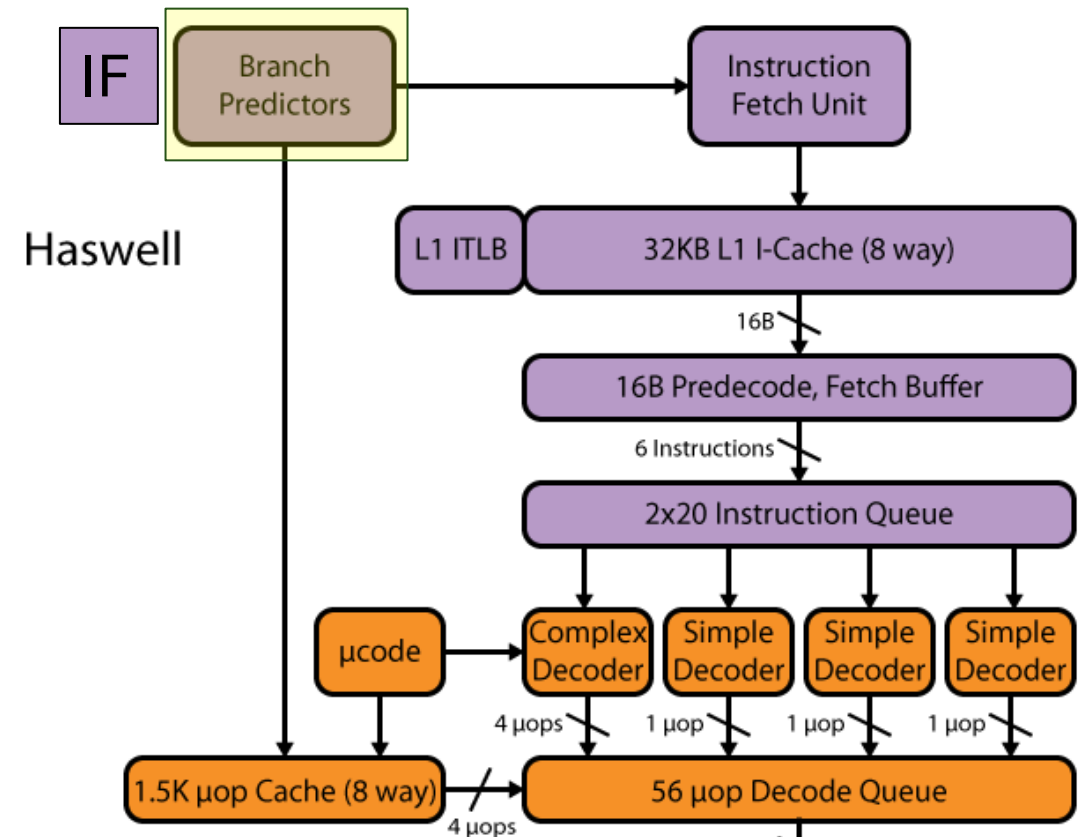
The effect on throughput of undoing computation is just like **stalling**

On Haswell, a **mispredicted** branch costs **15-20** cycles.

Supporting Speculative Execution

- Modern processors use **branch predictors** to increase the effectiveness of speculative execution

- The **fetch** stage dedicates hardware to predicting the outcomes of branches
- Modern branch predictors are accurate over **95%** of the time

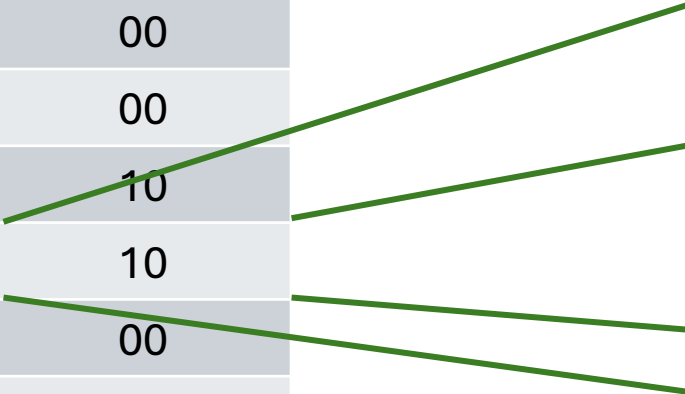


Simple Branch Prediction

- **Idea:** Hardware maintains a table mapping addresses of branch instructions to **predictions** of their outcomes.

Instruction address	Prediction
0x400c0c	00
0x400c1b	00
0x400c47	10
0x400cad	10
0x400cd0	00
0x400ced	00
0x400e37	01
0x400e4e	01
0x400e5c	11
0x400e61	10
0x400e72	10

A **prediction** is encoded as a **2-bit saturating counter**



Encoding		Meaning
1	1	Strongly taken
1	0	Weakly taken
0	1	Weakly not taken
0	0	Strongly not taken

A prediction counter is **updated** based on the **actual outcome** of the associated branch:

- Taken → increase counter.
- Not taken → decrease counter.

Outline

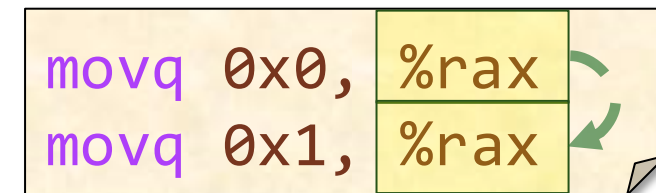
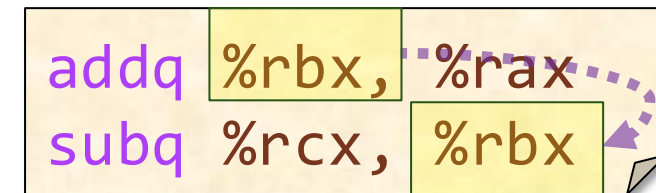
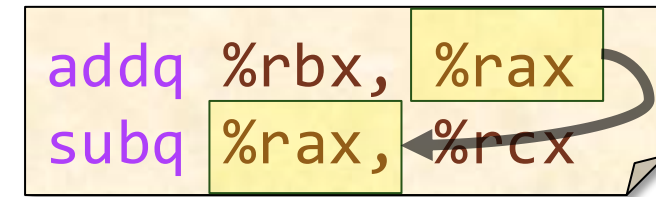
- x86-64 ISA Primer
 - Opcodes
 - Operands
- Overview of Modern Computer Architecture
 - Dealing with Structural Hazards
 - Dealing with Control Hazards
 - **Dealing with Data Hazards**

Sources of Data Hazards

- An instruction **i** can create a data hazard with a later instruction **j** due to a **dependence** between **i** and **j**.

- **True dependence (RAW)**: Instruction **i** writes a location that instruction **j** reads
- **Anti-dependence (WAR)**: Instruction **i** reads a location that instruction **j** writes
- **Output-dependence (WAW)**: Both **i** and **j** write to the same location

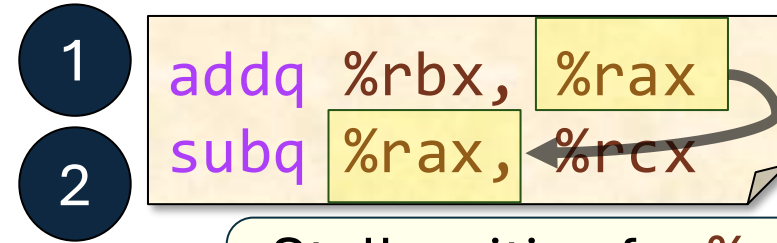
Examples



Bypassing

- **Bypassing** allows an instruction to read its arguments before they've been stored in a GPR

Example



Without bypassing

Instr. #	Cycle								
	1	2	3	4	5	6	7	8	9
1	IF	ID	EX	MA	WB				
2		IF	ID	ID	ID	EX	MA	WB	

Stall waiting for `%rax` to be written back to a register.

With bypassing

Instr. #	Cycle								
	1	2	3	4	5	6	7	8	9
1	IF	ID	EX	MA	WB				
2		IF	ID	EX	MA	WB			

Stall eliminated.

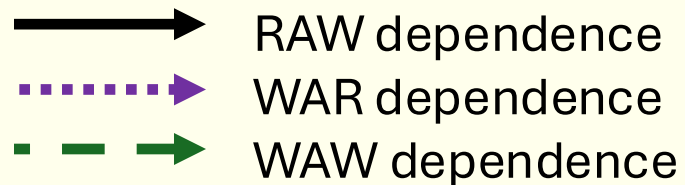
Data Dependencies: Example

- What else can the hardware do to exploit ILP?

Let's consider a larger code example with more data dependencies

Example instruction stream

#	Instruction	Latency
1	movsd (%rax), %xmm0	2
2	movsd (%rbx), %xmm2	5
3	mulsd %xmm0, %xmm2	3
4	addsd %xmm0, %xmm1	1
5	addsd %xmm1, %xmm1	1
6	mulsd %xmm1, %xmm0	3



Simplifying assumptions

- The issue stage issues **1** operation per cycle
- The processor has plenty of functional units for all operations.
- Ignore the non-execute stages of the pipeline
- Latencies are chosen to simplify the example

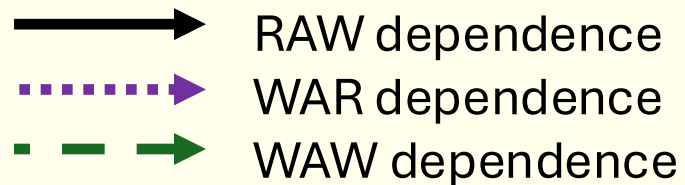
Data Dependencies: Example

- What else can the hardware do to exploit ILP?

Let's consider a larger code example with more data dependencies

Example instruction stream

#	Instruction	Latency
1	movsd (%rax), %xmm0	2
2	movsd (%rbx), %xmm2	5
3	mulsd %xmm0, %xmm2	3
4	addsd %xmm0, %xmm1	1
5	addsd %xmm1, %xmm1	1
6	mulsd %xmm1, %xmm0	3



Simplifying assumptions

- The issue stage issues **1** operation per cycle
- The processor has plenty of functional units for all operations.
- Ignore the non-execute stages of the pipeline
- Latencies are chosen to simplify the example

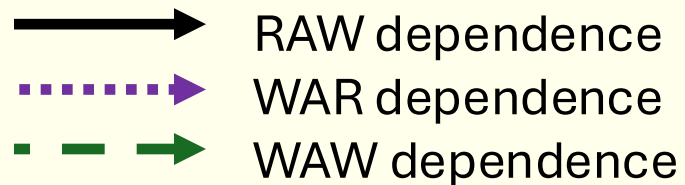
Data Dependencies: Example

- What else can the hardware do to exploit ILP?

Let's consider a larger code example with more data dependencies

Example instruction stream

#	Instruction	Latency
1	movsd (%rax), %xmm0	2
2	movsd (%rbx), %xmm2	5
3	mulsd %xmm0, %xmm2	3
4	addsd %xmm0, %xmm1	1
5	addsd %xmm1, %xmm1	1
6	mulsd %xmm1, %xmm0	3



Simplifying assumptions

- The issue stage issues **1** operation per cycle
- The processor has plenty of functional units for all operations.
- Ignore the non-execute stages of the pipeline
- Latencies are chosen to simplify the example

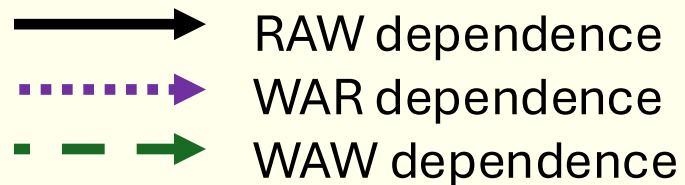
Data Dependencies: Example

- What else can the hardware do to exploit ILP?

Let's consider a larger code example with more data dependencies

Example instruction stream

#	Instruction	Latency
1	movsd (%rax), %xmm0	2
2	movsd (%rbx), %xmm2	5
3	mulsd %xmm0, %xmm2	3
4	addsd %xmm0, %xmm1	1
5	addsd %xmm1, %xmm1	1
6	mulsd %xmm1, %xmm0	3

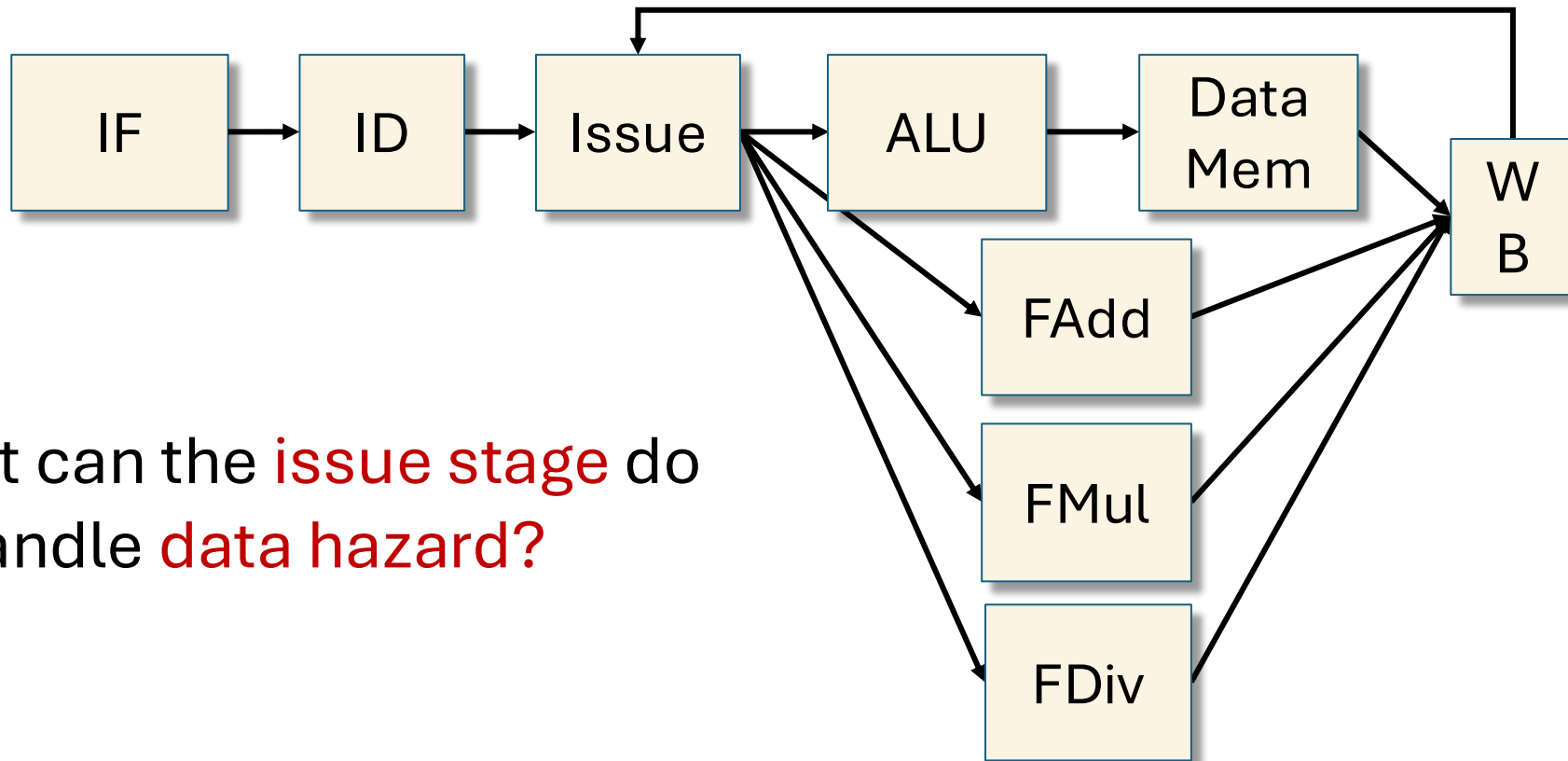


Simplifying assumptions

- The issue stage issues **1** operation per cycle
- The processor has plenty of functional units for all operations.
- Ignore the non-execute stages of the pipeline
- Latencies are chosen to simplify the example

Block Diagram of a Superscalar Pipeline

- The **issue** stage in the pipeline manages the functional units and handles scheduling of instructions.



What can the **issue stage** do to handle **data hazard**?

Example: In-Order Issue

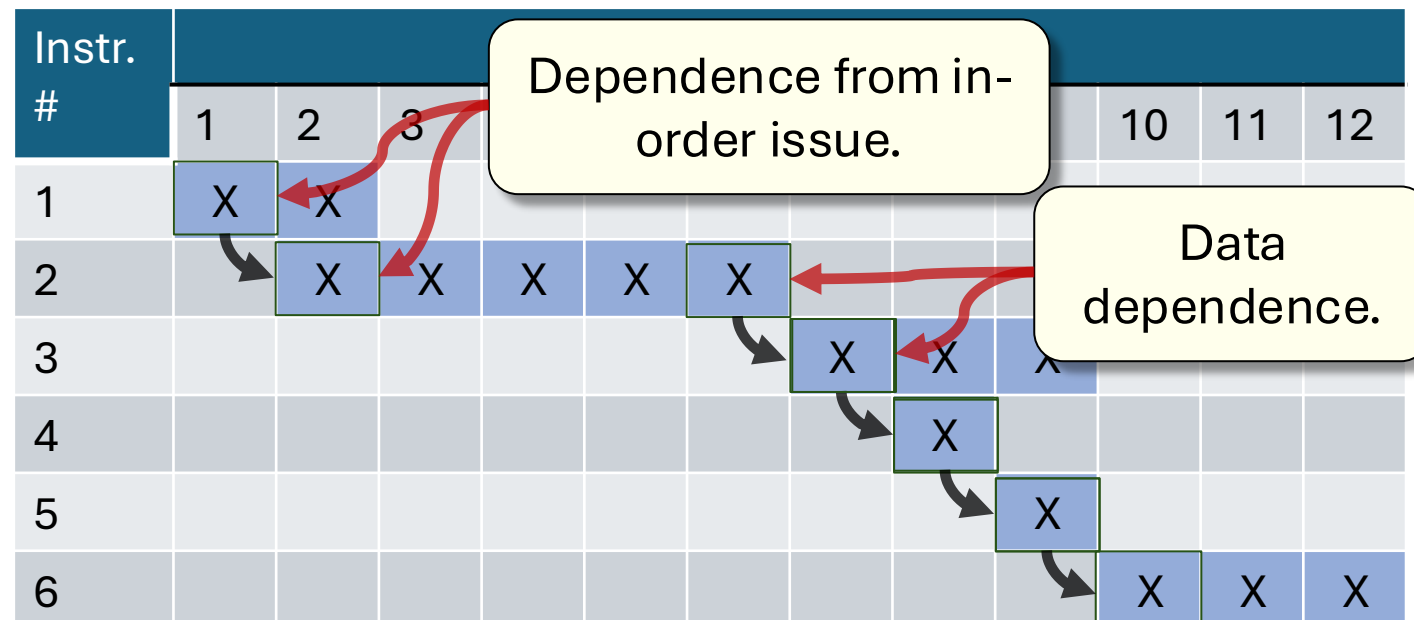
- If the hardware issues them **in order**, how long does it take?

Instruction stream

#	Instruction	Latency
1	movsd (%rax), %xmm0	2
2	movsd (%rbx), %xmm2	5
3	mulsd %xmm0, %xmm2	3
4	addsd %xmm0, %xmm1	1
5	addsd %xmm1, %xmm1	1
6	mulsd %xmm1, %xmm0	3

Instruction timing for use of functional units

Can the hardware do better?

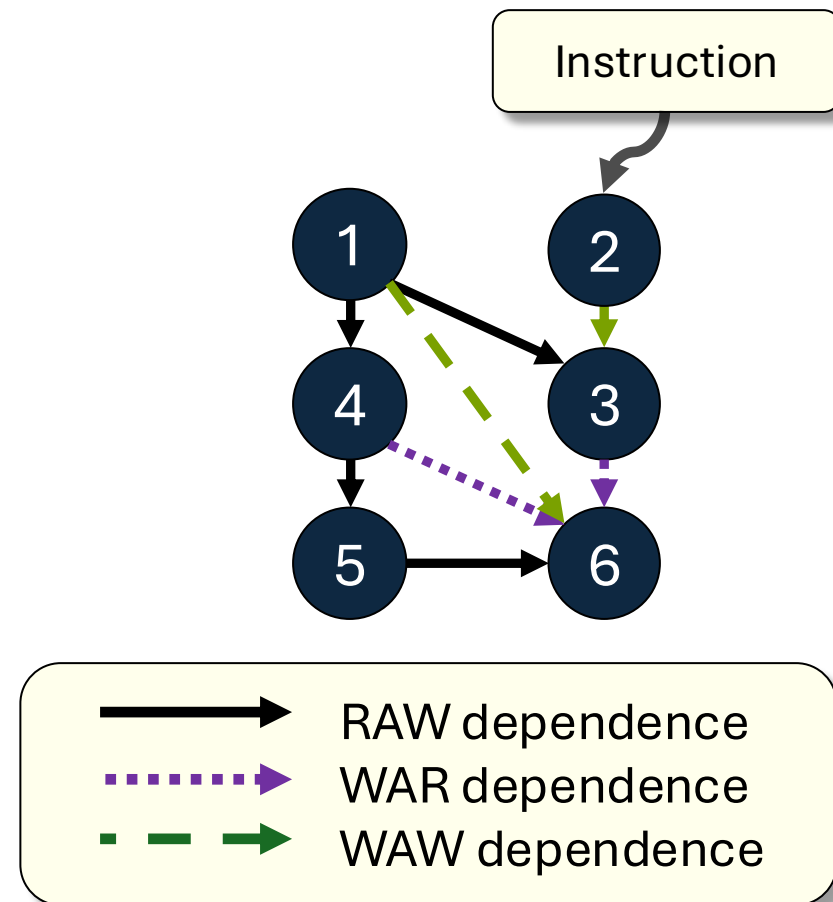


Data-Flow Graphs

- We can model the data dependencies between instructions as a **data-flow graph**.

Example instruction stream

#	Instruction	Latency
1	movsd (%rax), %xmm0	2
2	movsd (%rbx), %xmm2	5
3	mulsd %xmm0, %xmm2	3
4	addsd %xmm0, %xmm1	1
5	addsd %xmm1, %xmm1	1
6	mulsd %xmm1, %xmm0	3

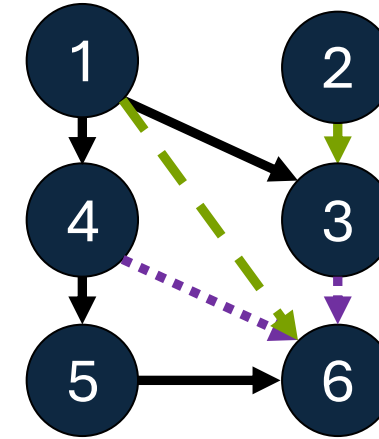


In-Order Issue, Revisited

Instruction
stream

#	Instruction	Latency
1	movsd (%rax), %xmm0	2
2	movsd (%rbx), %xmm2	5
3	mulsd %xmm0, %xmm2	3
4	addsd %xmm0, %xmm1	1
5	addsd %xmm1, %xmm1	1
6	mulsd %xmm1, %xmm0	3

Data-flow graph



Instruction timing for
use of functional units

Instr. #	Cycle											
	1	2	3	4	5	6	7	8	9	10	11	12
1	X	X										
2		X	X	X	X	X						
3							X	X	X			
								X				
									X			
										X	X	X

False dependences! Instruction 4
doesn't depend on instructions 2 or 3!

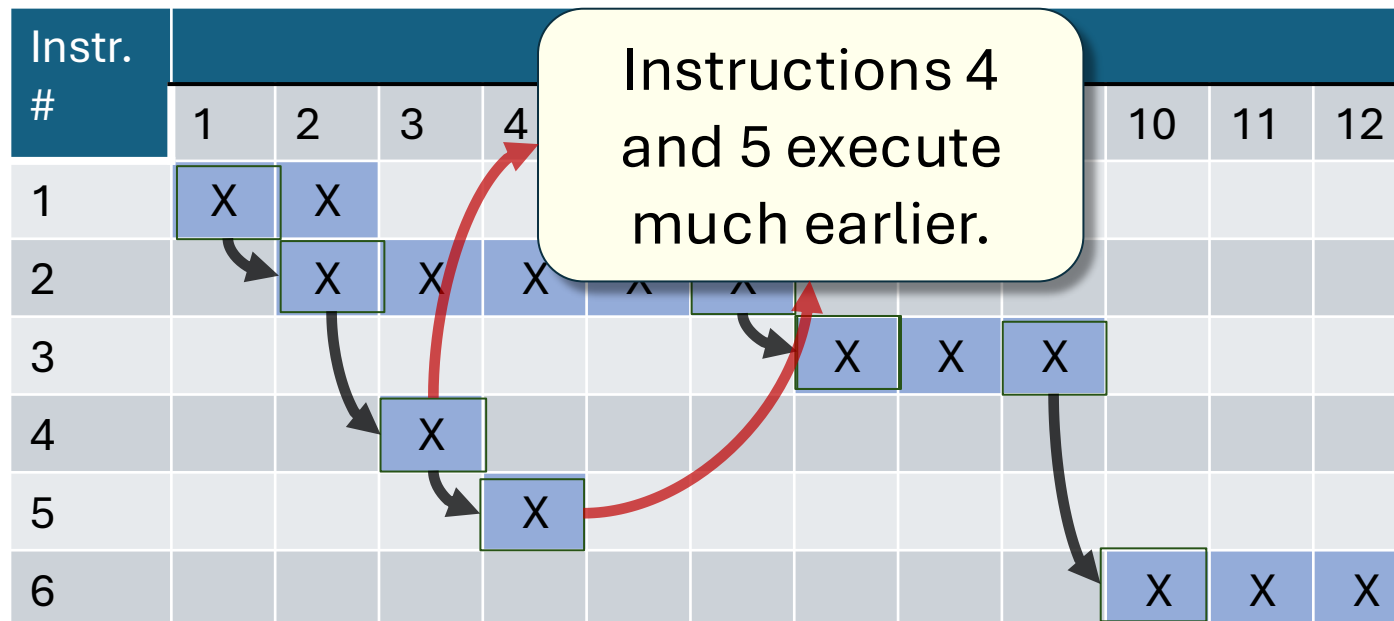
Example: Out-of-Order Execution

Instruction
stream

#	Instruction	Latency
1	movsd (%rax), %xmm0	2
2	movsd (%rbx), %xmm2	5
3	mulsd %xmm0, %xmm2	3
4	addsd %xmm0, %xmm1	1
5	addsd %xmm1, %xmm1	1
6	mulsd %xmm1, %xmm0	3

Idea: the hardware issues an instruction **as soon as dependencies are satisfied**

Instruction timing for
use of functional units



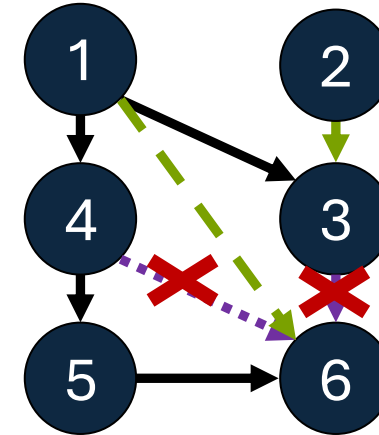
Can the hardware
do better?

Eliminating Name Dependencies

Instruction
stream

#	Instruction	Latency
1	movsd (%rax), %xmm0	2
2	movsd (%rbx), %xmm2	5
3	mulsd %xmm0 , %xmm2	3
4	addsd %xmm0 , %xmm1	1
5	addsd %xmm1, %xmm1	1
6	mulsd %xmm1, %xmm0	3

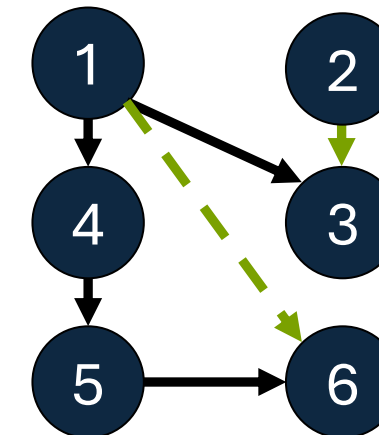
Data-flow graph



Idea: If the **name** of the **destination register** could be changed, then the WAR dependencies could be **eliminated**.

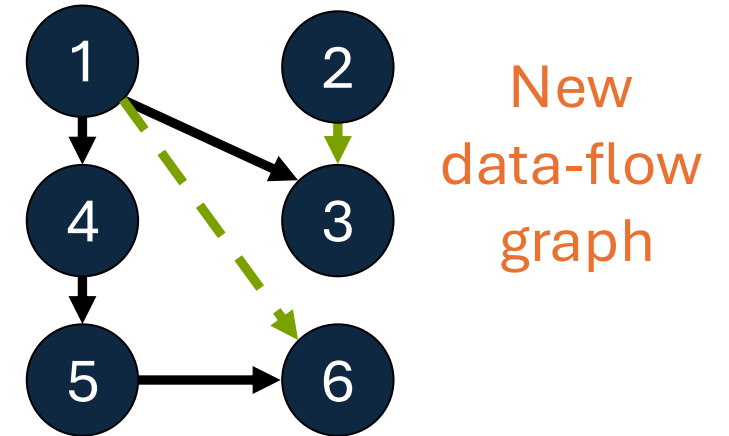
Instruction 6 no longer depends on long latency operations!

New data-flow graph



Effect of Eliminating Name Dependencies

- **Idea:** Let the hardware change destination register names **on the fly**



Combined with out-of-order execution, instructions 4, 5, and 6 can all execute earlier.

Instruction timing for use of functional units

Instr. #	1	2	3																
1	X	X																	
2		X	X	X	X	X													
3							X	X	X										
4			X																
5				X															
6					X	X	X												

Summary: Dealing with Hazards

- **Stalling:** freeze earlier pipeline stages
- **Bypassing:** route the data as soon as it is calculated to an earlier stage
- **Out-of-order execution:** execute a later instr. before an earlier one
- **Register renaming:** Remove dependence by changing its register operands
- **Speculation:** Guess the outcome and run; redo if guess is incorrect
- **Separate Functional Units + Superscalar:** structural hazards

Logistics

- Homework 3 on **Vectorization** will be released this Thursday
- Start working on it before Lecture 4 **Vectorization** next Tuesday
 - **Lecture 3 & 4** slides will be available today
- Quiz 1 will be on **October 22, Thursday, same room**